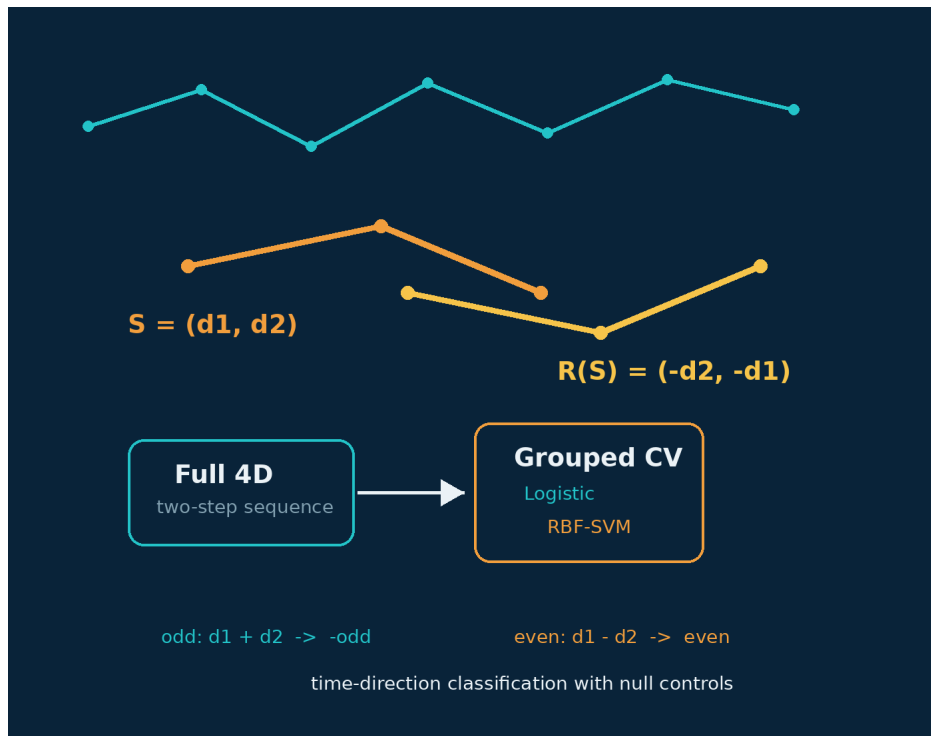




PHASE 7 STANDALONE HANDBOOK

Full-Sequence Time-Reversal Classification and Diagnostic Decomposition

Time Irreversibility in Cell Motility

A cell-by-cell scientific, mathematical, biophysical, Python, machine-learning, and decision-making record of Phase 7

VALIDATED PHASE SNAPSHOT

Executed notebook: notebooks/07_phase7_full_sequence_classification_MSC01.ipynb • Exact two-step sequences: 392 from 32 cells • Balanced classifier dataset: 784 examples • Logistic raw AUC: 0.622 with empirical paired-null $p \approx 0.005$ • Co-moving logistic AUC: 0.504 • Full 4D signal was not stronger than reversal-odd net displacement • RBF-SVM found no convincing extra full-sequence information • Automated tests: 9 passed.

Prepared as a standalone continuation companion to the Phase 0–4 Master Handbook and the Phase 5 and Phase 6 Standalone Handbooks

Document purpose, scope, and evidence standard

This standalone handbook reconstructs Phase 7 in enough detail that a fresh ChatGPT session, collaborator, or future version of the user can understand exactly what was done, why it was done, which numerical results were obtained, and which interpretations were deliberately rejected. It is designed as a continuation companion rather than a replacement for the Phase 0–4 Master Handbook and the standalone Phase 5 and Phase 6 handbooks.

DOCUMENTATION BOUNDARY

Phase 7 is an exploratory scientific training and portfolio phase. Machine learning is used as a diagnostic tool for time-direction information, not as a model-selection competition. The phase is reproducible and null-calibrated, but it is not framed as a publication-grade thermodynamic inference or a direct entropy-production measurement.

Evidence item	What it establishes
Executed Notebook 07	Authoritative Phase 7 cell order, code logic, validation structure, and saved analysis outputs.
Recorded output screenshots	Exact cross-validation tables, null summaries, decomposition results, RBF results, and final summary values used during interpretation.
Phase 6 standalone handbook	Immediate scientific context: exact two-step sequences, correct time reversal, reduced time-odd features, KDE asymmetry, and the negative null-calibrated Phase 6 result.
Automated test run	<code>`python -m pytest -q`</code> returned 9 passed in 0.56 s before the Phase 7 commit workflow.
README update through Phase 7	Project-level summary was revised to reflect full-sequence classification, grouped validation, odd/even decomposition, and nonlinear sensitivity.

EXACT STOPPING POINT

Phase 7 is scientifically complete. The full two-step sequence was tested with an interpretable linear classifier and one pre-specified nonlinear sensitivity model. The raw linear signal was traced to net directional displacement, and no convincing extra full-sequence ordering signal was found. Do not add more classifiers merely to search for a positive result.

[↑ Back to Contents](#)

Contents

- [1. Phase 7 handoff from Phase 6](#)
- [2. Mathematical, statistical, and machine-learning foundations](#)
- [3. Data inputs, invariants, and Phase 7 design](#)
- [4. Cells 1–12: full-sequence and grouped-validation foundation](#)
- [5. Cells 13–20: baseline linear classifier](#)
- [6. Cells 21–27: paired time-reversal classification null](#)
- [7. Cells 28–34: reversal-odd / reversal-even decomposition](#)
- [8. Cells 35–40: nonlinear RBF sensitivity and fold diagnostics](#)
- [9. Cells 41–44: RBF paired-null validation](#)
- [10. Cells 45–46: final numerical and scientific conclusion](#)
- [11. Integrated Phase 7 results and interpretation](#)
- [12. Decision logic and project mindset](#)
- [13. Python, statistics, and machine-learning concepts learned](#)
- [14. Reproducibility, testing, and Git workflow](#)
- [15. Phase 8 handoff and next decision space](#)
- [Appendix A. Key Phase 7 code listings](#)
- [Appendix B. Exact numerical result ledger](#)
- [Appendix C. Glossary and notation](#)
- [Appendix D. New-chat handoff prompt](#)

Every major section deliberately starts on a new page. Unused space at the end of a section is intentional so that section boundaries remain visually obvious.

1. Phase 7 handoff from Phase 6

Phase 6 established the correct ordered time-reversal operation for exact two-step trajectories and tested an interpretable reduced feature representation. That representation used change in step magnitude and signed turning information, but its observed KDE asymmetry did not exceed a time-reversal-symmetrized finite-sample null. Phase 7 therefore asked a narrower follow-up question: was the Phase 6 negative result caused by compressing the original four-dimensional sequence too aggressively?

$$S = (\Delta x_1, \Delta y_1, \Delta x_2, \Delta y_2)$$
$$R(S) = (-\Delta x_2, -\Delta y_2, -\Delta x_1, -\Delta y_1)$$

PHASE 7 QUESTION

Does the complete four-dimensional two-step sequence contain forward-versus-reversed information that was lost when Phase 6 reduced each sequence to two hand-designed time-odd features?

Inherited validated fact	Phase 7 consequence
392 exact two-step sequences from 32 cells exist in both raw and co-moving representations.	Use all 392 sequences because the full 4D representation does not require a defined turning angle.
Raw and co-moving sequence identities are structurally paired.	Use identical cells, sequence identities, folds, and null randomizations for fair comparison.
Correct time reversal is $R(S)=(-\Delta r_2,-\Delta r_1)$.	Create an exact reversed counterpart for every observed forward sequence.
Phase 6 reduced-feature null was negative.	Do not assume full-sequence classification will be positive; treat both outcomes as informative.
Common population motion is present but not uniquely attributable to technical drift.	Keep raw and co-moving analyses in parallel and interpret differences conservatively.

KEY METHODOLOGICAL UPGRADE

Instead of estimating a probability density directly in four dimensions, Phase 7 recasts time-direction detection as a forward-versus-reversed classification problem with cell-grouped cross-validation and paired null controls.

[↑ Back to Contents](#)

2. Mathematical, statistical, and machine-learning foundations

2.1 Forward and reversed sequence classes

Each observed sequence contributes two classifier examples: the observed forward orientation and its exact time-reversed orientation. The labels are balanced by construction: forward = 1 and reversed = 0. With 392 original sequences, each representation contains 784 examples.

$$392 \text{ forward} + 392 \text{ reversed} = 784 \text{ classifier examples}$$

2.2 Why classification can replace direct 4D density estimation

Directly estimating $p(S)$ and $p(R(S))$ in four dimensions would be data-hungry for only a few hundred sequences. A classifier instead asks whether a decision rule trained on some cells can rank or separate forward and reversed examples from held-out cells. This tests detectable information without requiring an explicit four-dimensional KDE.

2.3 Biological grouping and information leakage

Sequences from the same cell are not independent. A naive random split could place highly related examples from one cell in both training and testing, inflating apparent generalization. GroupKFold uses `cell_id` only as a grouping variable so that every example from a test cell is absent from training. The cell identifier is never used as a predictive feature.

LEAKAGE RULE

All forward and reversed examples belonging to one biological cell must remain in the same cross-validation fold. Preprocessing must also be fitted inside each training fold.

2.4 Logistic regression and the linear baseline

$$z = \beta_0 + \beta_1 \Delta x_1 + \beta_2 \Delta y_1 + \beta_3 \Delta x_2 + \beta_4 \Delta y_2$$

Logistic regression converts this linear score into a forward-class probability. Its value here is interpretability: if a simple linear time-direction rule generalizes across cells, we can decompose that rule mathematically before considering nonlinear models.

2.5 Standardization inside a Pipeline

$$x_{\text{scaled}} = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$$

StandardScaler is placed inside a scikit-learn Pipeline so that the mean and standard deviation are learned from training cells only. Scaling the entire dataset before cross-validation would leak information from held-out cells.

2.6 Evaluation metrics

ROC AUC measures ranking quality across decision thresholds. AUC=0.5 is chance-level ranking; values above 0.5 indicate correct-direction discrimination. Balanced accuracy averages class-wise recall and is approximately 0.5 at chance for the perfectly balanced Phase 7 dataset.

2.7 Paired time-reversal null

The null does not shuffle labels independently. Instead, each exact pair $\{S_i, R(S_i)\}$ randomly swaps which orientation is treated as pseudo-forward. This preserves sequence values, pair structure, cell membership, sample size, class balance, and grouped cross-validation while removing systematic time direction in expectation.

2.8 Empirical upper-tail probability

$$p_{\text{emp}} = [1 + \#\{\text{null} \geq \text{observed}\}] / (N_{\text{null}} + 1)$$

The +1 correction prevents a zero p-value when the observed statistic exceeds every simulated null replicate. These p-values are treated as exploratory empirical tail probabilities, not as a license for publication-style significance claims.

2.9 Reversal-odd and reversal-even coordinates

$$o = d_1 + d_2, \quad e = d_1 - d_2$$

$$\text{under } R: \quad o \rightarrow -o, \quad e \rightarrow e$$

The odd coordinate is exactly the two-frame net displacement $r(t+2)-r(t)$. A linear classifier can exploit this directional quantity without using richer sequence ordering. The even coordinate is invariant under reversal and therefore contains no standalone class information.

2.10 Why one nonlinear model is enough

An RBF support-vector classifier was used as a single pre-specified nonlinear sensitivity model. Its purpose was to detect nonlinear interactions, including interactions between odd and even coordinates, without launching a model zoo or tuning search. No neural network, random-forest sweep, or post-hoc hyperparameter chase was performed.

INTERPRETATION BOUNDARY

Classifier performance is not entropy production. A positive AUC can arise from directional motion, collective translation, technical drift, or other non-thermodynamic structure. Thermodynamic-arrow claims remain deferred.

[↑ Back to Contents](#)

3. Data inputs, invariants, and Phase 7 design

Item	Phase 7 value / decision
Raw input	data/MS01_tracks_clean.csv
Co-moving input	data/MS01_tracks_comoving.csv
Selected track columns	cell_id, frame, t_min, x_um, y_um
Frame interval	20 min
Exact two-step sequences	392 sequences from 32 cells in each representation
Full feature vector	[dx1_um, dy1_um, dx2_um, dy2_um]
Time reverse	[-dx2_um, -dy2_um, -dx1_um, -dy1_um]
Classifier dataset	784 examples = 392 forward + 392 reversed
Cross-validation	5-fold GroupKFold by cell_id
Linear model	StandardScaler → LogisticRegression(max_iter=2000)
Linear null	200 paired orientation-randomization replicates
Odd coordinate	$d1 + d2$ = two-frame net displacement
Even coordinate	$d1 - d2$ = reversal-invariant step difference
Nonlinear model	StandardScaler → SVC(kernel="rbf", C=1.0, gamma="scale")
RBF null	200 paired orientation-randomization replicates
Random seed family	2026, 2027, 2028 across defined analyses
Interpretation	Exploratory, grouped, null-calibrated; no entropy-production claim

DESIGN DISCIPLINE

Phase 7 deliberately stopped after one interpretable linear model, one mathematical decomposition, and one predefined nonlinear sensitivity model. More models would increase result-chasing risk without improving the scientific question.

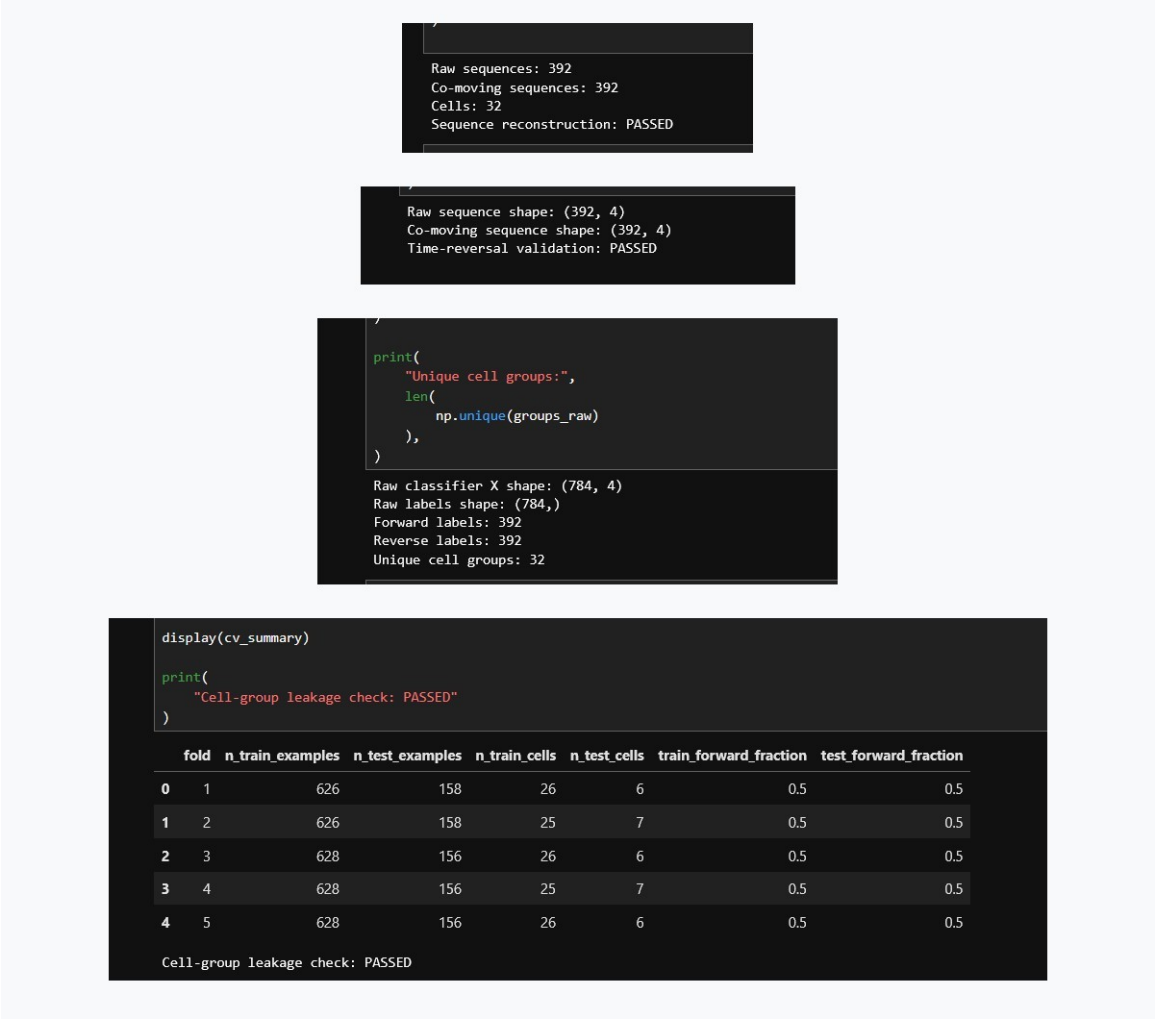


Figure 1. Phase 7 construction and grouped-validation checkpoints: exact sequence counts and shapes, balanced classifier dataset, and five cell-grouped folds with 50/50 labels.

[↑ Back to Contents](#)

4. Cells 1–12: full-sequence and grouped-validation foundation

This block reconstructs the exact full-sequence analysis object and establishes a leakage-safe classification dataset before any model is fitted.

Cell 1 — Phase title and scientific goal

Type / role	Markdown / scope definition
Code or content summary	Defined Phase 7 as a full-sequence forward-versus-reversed classification test using the complete 4D two-step displacement representation.
Stored output	No executable output.
Interpretation / decision	Locked the central question: did Phase 6 lose detectable time-direction information by reducing the sequence?
Concepts taught / reinforced	Scientific scoping; explicit null-compatible objective; interpretation boundary.

Cell 2 — Preventing information leakage

Type / role	Markdown / validation design
Code or content summary	Explained why repeated sequences from the same cell cannot be split naively and why all examples from one cell must stay in one fold.
Stored output	No executable output.
Interpretation / decision	Established cell-level grouping as a non-negotiable validation rule.
Concepts taught / reinforced	Biological dependence; train/test leakage; grouped cross-validation.

Cell 3 — Imports and constants

Type / role	Code / environment setup
Code or content summary	Ran <code>`_path.py`</code> ; imported NumPy, pandas, Matplotlib, LogisticRegression, metrics, GroupKFold, Pipeline, StandardScaler, validated trajectory helpers; set frame interval, seed, and five folds.
Stored output	Printed Phase 7 environment information and project root.
Interpretation / decision	Reused validated project infrastructure and fixed key analysis constants before fitting models.
Concepts taught / reinforced	Imports; constants; reproducibility; modular reuse.

Cell 4 — Load raw and co-moving trajectories

Type / role	Code / data loading
-------------	---------------------

Code or content summary	Loaded the cleaned raw and co-moving CSV files, selected canonical columns, sorted by cell/frame, copied, and validated.
Stored output	Validated dataframes loaded without structural errors.
Interpretation / decision	Kept raw and co-moving data parallel from the beginning.
Concepts taught / reinforced	pandas indexing; sorting; defensive copies; validation.

Cell 5 – Build exact two-step sequences

Type / role	Code / sequence function
Code or content summary	Reused validated exact one-frame displacements, merged first and second steps on the exact middle frame, and rejected nonconsecutive chains.
Stored output	Function definition only.
Interpretation / decision	Prevented the earlier row-shift lag bug from reappearing in sequence analysis.
Concepts taught / reinforced	Functions; DataFrame merge; one-to-one validation; temporal continuity.

Cell 6 – Construct raw and co-moving sequences

Type / role	Code / sequence construction
Code or content summary	Built exact sequences in both representations and compared structural identity.
Stored output	Raw sequences: 392; Co-moving sequences: 392; Cells: 32; reconstruction PASSED.
Interpretation / decision	Confirmed a perfectly paired sequence backbone for all later comparisons.
Concepts taught / reinforced	Assertions; paired data structures; invariant checking.

Cell 7 – Sequence arrays and reversal validation

Type / role	Code / mathematical validation
Code or content summary	Converted each sequence to a NumPy row [dx1,dy1,dx2,dy2], generated exact time reverses, and tested double reversal.
Stored output	Raw shape (392,4); Co-moving shape (392,4); time-reversal validation PASSED.
Interpretation / decision	Verified the central transformation before classification.
Concepts taught / reinforced	NumPy arrays; column stacking; involution tests.

Cell 8 – Constructing the classification dataset

Type / role	Markdown / label design
-------------	-------------------------

Code or content summary	Defined forward label 1 and reversed label 0; explained perfect class balance and reuse of cell_id only for grouping.
Stored output	No executable output.
Interpretation / decision	Separated predictive features from grouping metadata.
Concepts taught / reinforced	Supervised-learning dataset design; class labels; grouping variables.

Cell 9 – Build classification dataset function

Type / role	Code / reusable helper
Code or content summary	Stacked forward and reversed arrays vertically, concatenated labels and cell groups, and validated dimensions.
Stored output	Function definition only.
Interpretation / decision	Created one deterministic helper used in observed and null analyses.
Concepts taught / reinforced	np.vstack; np.concatenate; shape validation; reusable functions.

Cell 10 – Construct raw and co-moving classifier sets

Type / role	Code / dataset construction
Code or content summary	Built X, y, and groups for both representations from the same 392 sequence identities.
Stored output	X shape (784,4); y shape (784,); 392 forward, 392 reverse, 32 groups.
Interpretation / decision	Confirmed exact balance and shared grouping before modeling.
Concepts taught / reinforced	Feature matrix; target vector; group vector; array shapes.

Cell 11 – Validate classifier pairing

Type / role	Code / assertions
Code or content summary	Checked dimensions, label counts, identical labels/groups across raw and co-moving, and 32 unique cell groups.
Stored output	Classification dataset validation: PASSED.
Interpretation / decision	Locked the structural invariants used by every classifier.
Concepts taught / reinforced	Assertions; array equality; sanity checks.

Cell 12 – Create grouped CV folds

Type / role	Code / leakage audit
Code or content summary	Used GroupKFold(n_splits=5); verified no train/test cell overlap; recorded example and cell counts plus class fractions.

Stored output	Fold train/test examples: 626/158, 626/158, 628/156, 628/156, 628/156; test cells 6/7/6/7/6; all label fractions 0.5; leakage check PASSED.
Interpretation / decision	Demonstrated leakage-safe five-fold evaluation with exact class balance.
Concepts taught / reinforced	GroupKFold; enumerate; index arrays; np.unique/intersect1d; fold diagnostics.

[↑ Back to Contents](#)

5. Cells 13–20: baseline linear classifier

The first machine-learning analysis uses a deliberately simple and interpretable linear model. The goal is not maximum predictive performance; it is a leakage-safe baseline whose structure can later be decomposed.

Cell 13 – Baseline linear time-direction classifier

Type / role	Markdown / model rationale
Code or content summary	Defined logistic regression on the full 4D sequence and emphasized scaling inside the cross-validation pipeline.
Stored output	No executable output.
Interpretation / decision	Chose interpretability and low complexity as the first diagnostic.
Concepts taught / reinforced	Logistic regression; linear decision score; preprocessing leakage.

Cell 14 – Create the linear Pipeline

Type / role	Code / model definition
Code or content summary	Created StandardScaler followed by LogisticRegression(max_iter=2000).
Stored output	VS Code rendered the Pipeline diagram: StandardScaler → LogisticRegression.
Interpretation / decision	No training occurred yet; the cell only defined the estimator.
Concepts taught / reinforced	scikit-learn Pipeline; transformers; estimators; max_iter.

Cell 15 – Define evaluation metrics

Type / role	Markdown / metrics
Code or content summary	Defined ROC AUC and balanced accuracy, both with chance near 0.5 in this balanced design.
Stored output	No executable output.
Interpretation / decision	Prevented later interpretation from relying on one arbitrary classification threshold.
Concepts taught / reinforced	ROC AUC; balanced accuracy; threshold-free ranking.

Cell 16 – Grouped classifier evaluator

Type / role	Code / reusable evaluation
Code or content summary	For each GroupKFold split, fitted the model only on training cells, computed forward probabilities, predicted labels, AUC, balanced accuracy, and rechecked group separation.
Stored output	Function definition only.
Interpretation / decision	Centralized leakage-safe evaluation for raw, co-moving, and null runs.

Concepts taught / reinforced	fit/predict/predict_proba; fold loops; metric functions.
------------------------------	--

Cell 17 – Evaluate raw trajectories

Type / role	Code / observed linear result
Code or content summary	Ran five grouped folds on raw full-4D sequences.
Stored output	Fold AUCs: 0.637478, 0.662554, 0.615220, 0.483892, 0.710634; mean later = 0.621956.
Interpretation / decision	Raw data showed apparent above-chance linear forward/reverse information, but fold heterogeneity required null calibration.
Concepts taught / reinforced	Cross-validation variability; out-of-sample performance.

Cell 18 – Evaluate co-moving trajectories

Type / role	Code / observed linear result
Code or content summary	Ran the identical grouped evaluation on co-moving full-4D sequences.
Stored output	Fold AUCs: 0.520429, 0.611601, 0.497041, 0.389944, 0.506739; mean later = 0.504151.
Interpretation / decision	Common-motion subtraction reduced the linear signal toward chance.
Concepts taught / reinforced	Paired representation comparison; sensitivity analysis.

Cell 19 – Summarize observed performance

Type / role	Code / result table
Code or content summary	Computed mean and sample standard deviation of AUC and balanced accuracy across the five folds.
Stored output	Raw: AUC 0.621956 ± 0.084941 , balanced accuracy 0.614395 ± 0.099256 . Co-moving: AUC 0.504151 ± 0.080743 , balanced accuracy 0.519994 ± 0.090315 .
Interpretation / decision	Established the observed statistics to be judged by a null model.
Concepts taught / reinforced	DataFrame construction; mean; std(ddof=1).

Cell 20 – Baseline classifier validation

Type / role	Code / assertions
Code or content summary	Checked five folds per representation, metric bounds [0,1], and finite summary statistics.
Stored output	Baseline classifier validation: PASSED.
Interpretation / decision	Confirmed computational validity before null simulation.

Concepts taught / reinforced

Range checks; np.isfinite; validation gates.

```
display(
  raw_cv_results
)
```

	fold	n_train_examples	n_test_examples	n_test_cells	roc_auc	balanced_accuracy
0	1	626	158	6	0.637478	0.613924
1	2	626	158	7	0.662554	0.772152
2	3	628	156	6	0.615220	0.576923
3	4	628	156	7	0.483892	0.500000
4	5	628	156	6	0.710634	0.608974

```
display(
  observed_performance
)
```

	representation	mean_roc_auc	std_roc_auc	mean_balanced_accuracy	std_balanced_accuracy
0	raw	0.621956	0.084941	0.614395	0.099256
1	comoving	0.504151	0.080743	0.519994	0.090315

```
display(
  null_summary
)
```

	metric	observed	null_mean	null_q95	empirical_p	above_null_q95
0	raw_roc_auc	0.621956	0.500347	0.555552	0.004975	True
1	comoving_roc_auc	0.504151	0.502024	0.559142	0.512438	False
2	raw_minus_comoving_auc	0.117805	-0.001677	0.033821	0.004975	True

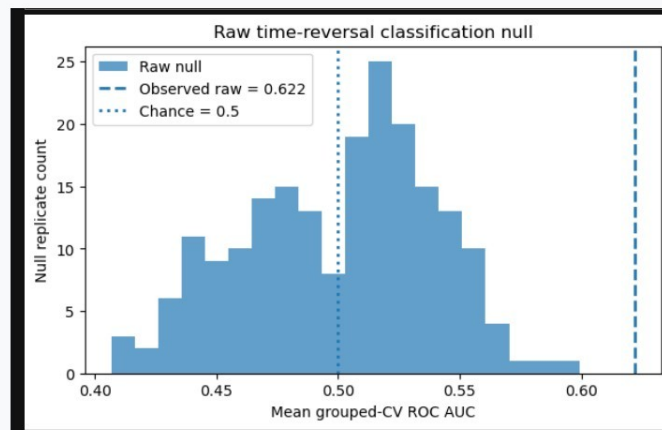


Figure 2. Baseline linear classification and paired-null outputs: raw and co-moving fold tables, observed performance, null summary, and the raw null histogram.

[↑ Back to Contents](#)

6. Cells 21–27: paired time-reversal classification null

A raw AUC above 0.5 is not sufficient. This block constructs a pair-preserving null that keeps the exact sequence/reverse structure and cell grouping while randomizing temporal orientation.

Cell 21 — Time-reversal-symmetrized classification null

Type / role	Markdown / null rationale
Code or content summary	Specified pair-level random orientation swaps instead of independent label shuffling.
Stored output	No executable output.
Interpretation / decision	Preserved the most important geometry and dependence structure of the classifier dataset under the null.
Concepts taught / reinforced	Null-model design; exchangeability at the pair level; paired controls.

Cell 22 — Randomize pair orientation

Type / role	Code / null helper
Code or content summary	Used a boolean swap mask to exchange forward and reversed members for selected sequence pairs while leaving unselected pairs unchanged.
Stored output	Function definition only.
Interpretation / decision	Created a reversible and interpretable null transformation.
Concepts taught / reinforced	Boolean masks; copies; advanced indexing.

Cell 23 — Validate the null transformation

Type / role	Code / unit-style check
Code or content summary	Generated a seeded random mask and verified pairwise keep/swap behavior row by row.
Stored output	Pair-orientation null transformation: PASSED; swapped count approximately half of 392.
Interpretation / decision	Confirmed the null preserves values and only changes temporal identity.
Concepts taught / reinforced	Random generators; boolean conditions; np.allclose.

Cell 24 — Run repeated null classifications

Type / role	Code / simulation
Code or content summary	Ran 200 paired null replicates using the same swap mask for raw and co-moving; each replicate used five grouped folds for both representations.
Stored output	Null replicates: 200; stored raw/comoving AUC, balanced accuracy, and raw-minus-comoving AUC.

Interpretation / decision	Generated the finite-sample background distribution for the exact classifier design.
Concepts taught / reinforced	Monte Carlo simulation; nested loops; paired randomization.

Cell 25 – Compare observed AUC with null

Type / role	Code / null calibration
Code or content summary	Computed null mean, null 95th percentile, one-sided empirical upper-tail probability with +1 correction, and above-q95 flags.
Stored output	Raw AUC: observed 0.621956, null mean 0.500347, q95 0.555552, p=0.004975, above q95. Co-moving: 0.504151, null mean 0.502024, q95 0.559142, p=0.512438. Raw-minus-comoving: 0.117805, q95 0.033821, p=0.004975.
Interpretation / decision	Raw linear discrimination is unlikely under the paired null; co-moving is fully compatible with it.
Concepts taught / reinforced	Empirical p-values; quantiles; paired contrasts.

Cell 26 – Visualize raw observed vs null

Type / role	Code / figure
Code or content summary	Plotted 200 raw null mean-AUC values, the observed raw AUC, and chance=0.5.
Stored output	Observed raw AUC lies far to the right of the simulated null distribution.
Interpretation / decision	Made the null calibration visually transparent rather than relying only on a p-value.
Concepts taught / reinforced	Histograms; reference lines; visual diagnostics.

Cell 27 – Validate null analysis

Type / role	Code / assertions
Code or content summary	Checked 200 replicates, finite values, bounded metrics, and valid empirical p-values.
Stored output	Time-reversal null validation: PASSED.
Interpretation / decision	Closed the linear null stage before interpreting mechanism.
Concepts taught / reinforced	Simulation validation; metric bounds; stopping gates.

CRITICAL INTERPRETATION

The raw logistic result is genuinely above this paired null, but that still does not show rich temporal ordering. The next question is what mathematical feature the linear classifier is actually using.

[↑ Back to Contents](#)

7. Cells 28–34: reversal-odd / reversal-even decomposition

This diagnostic converts the full sequence into coordinates with known transformation rules under time reversal. It is the key step that explains the apparently significant raw linear classifier.

Cell 28 – Decompose full sequence by reversal parity

Type / role	Markdown / mathematical diagnostic
Code or content summary	Defined odd coordinate $o=d1+d2$ and even coordinate $e=d1-d2$; noted $o=r(t+2)-r(t)$, the net two-frame displacement.
Stored output	No executable output.
Interpretation / decision	Predicted that a purely linear raw signal might be explained entirely by net directional displacement.
Concepts taught / reinforced	Symmetry decomposition; time-odd/time-even observables.

Cell 29 – Compute odd and even coordinates

Type / role	Code / feature transform
Code or content summary	Sliced the first and second 2D steps from the 4D array and returned their sum and difference.
Stored output	Function definition only.
Interpretation / decision	Created an exact invertible linear reparameterization for diagnostic classification.
Concepts taught / reinforced	NumPy slicing; vectorized addition/subtraction.

Cell 30 – Validate transformation under reversal

Type / role	Code / mathematical assertions
Code or content summary	Computed odd/even coordinates for forward and reversed arrays and tested odd sign-flip plus even invariance in both representations.
Stored output	Reversal-coordinate transformation: PASSED; odd and even shapes (392,2).
Interpretation / decision	Confirmed the symmetry algebra numerically before using the features.
Concepts taught / reinforced	Symmetry tests; shape checks; np.allclose.

Cell 31 – Build odd-only and even-only classifier sets

Type / role	Code / dataset construction
Code or content summary	Constructed balanced 784×2 classification datasets for raw/comoving odd and even coordinates.
Stored output	All four feature matrices had shape (784,2).

Interpretation / decision	Enabled direct comparison of full, odd-only, and even-only classification.
Concepts taught / reinforced	Feature-space construction; paired labels/groups.

Cell 32 – Evaluate the two subspaces

Type / role	Code / grouped classification
Code or content summary	Ran the same leakage-safe logistic evaluator on all four odd/even datasets.
Stored output	Results stored for summary; no direct cell output required.
Interpretation / decision	Held the model and folds fixed so only feature-space content changed.
Concepts taught / reinforced	Controlled comparisons; reusable evaluators.

Cell 33 – Compare full, odd, and even models

Type / role	Code / diagnostic summary
Code or content summary	Tabulated mean AUC and balanced accuracy for raw/co-moving full 4D, odd net displacement, and even step difference.
Stored output	Raw: full 0.621956, odd 0.621925, even 0.500000. Co-moving: full 0.504151, odd 0.504087, even 0.500000. Balanced accuracies full=odd and even=0.5.
Interpretation / decision	Nearly all linear raw performance is carried by net displacement; the full sequence adds essentially nothing.
Concepts taught / reinforced	Ablation/decomposition; symmetry-based interpretation.

Cell 34 – Validate reversal subspaces

Type / role	Code / exact equality checks
Code or content summary	Verified that forward and reversed even-coordinate matrices are identical and that summary metrics are finite.
Stored output	Reversal-subspace diagnostic: PASSED.
Interpretation / decision	Established that even-only classification must be chance by construction.
Concepts taught / reinforced	Identity under symmetry; exact diagnostic expectations.

CENTRAL PHASE 7 FINDING

The significant raw linear AUC is not evidence that logistic regression learned complex two-step ordering. Full-4D and odd-only AUC differ by only ~ 0.00003 . The classifier is effectively reading the two-frame net displacement.

```
display(
  subspace_summary
)
```

	representation	feature_space	mean_roc_auc	mean_balanced_accuracy
0	raw	full_4D	0.621956	0.614395
1	raw	odd_net_displacement	0.621925	0.614395
2	raw	even_step_difference	0.500000	0.500000
3	comoving	full_4D	0.504151	0.519994
4	comoving	odd_net_displacement	0.504087	0.519994
5	comoving	even_step_difference	0.500000	0.500000

```
display(
  rbf_observed_summary
)
```

	representation	feature_space	mean_roc_auc	mean_balanced_accuracy
0	raw	full_4D	0.557226	0.525625
1	raw	odd_net_displacement	0.565286	0.541042
2	comoving	full_4D	0.348291	0.372704
3	comoving	odd_net_displacement	0.371427	0.402077

	representation	feature_space	fold	n_test_cells	roc_auc	balanced_accuracy
0	raw	full_4D	1	6	0.701410	0.651899
1	raw	full_4D	2	7	0.414357	0.354430
2	raw	full_4D	3	6	0.599934	0.589744
3	raw	full_4D	4	7	0.487179	0.512821
4	raw	full_4D	5	6	0.583251	0.519231
5	raw	odd_net_displacement	1	6	0.686188	0.639241
6	raw	odd_net_displacement	2	7	0.402980	0.354430
7	raw	odd_net_displacement	3	6	0.631986	0.602564
8	raw	odd_net_displacement	4	7	0.504602	0.551282
9	raw	odd_net_displacement	5	6	0.600674	0.557692
10	comoving	full_4D	1	6	0.392085	0.392405
11	comoving	full_4D	2	7	0.200288	0.253165
12	comoving	full_4D	3	6	0.513314	0.525641
13	comoving	full_4D	4	7	0.331690	0.358974
14	comoving	full_4D	5	6	0.304076	0.333333
15	comoving	odd_net_displacement	1	6	0.435187	0.436709
16	comoving	odd_net_displacement	2	7	0.209422	0.253165
17	comoving	odd_net_displacement	3	6	0.564267	0.551282
18	comoving	odd_net_displacement	4	7	0.336785	0.371795
19	comoving	odd_net_displacement	5	6	0.311473	0.397436

```
display(
  rbf_null_summary
)
```

		metric	observed	null_mean	null_q95	empirical_upper_p	above_null_q95
0		raw_full_auc	0.557226	0.497950	0.565600	0.074627	False
1		raw_odd_auc	0.565286	0.499204	0.565278	0.054726	True
2		raw_full_minus_odd	-0.008060	-0.001254	0.046895	0.601990	False
3		comoving_full_auc	0.348291	0.498815	0.557063	1.000000	False
4		comoving_odd_auc	0.371427	0.501954	0.565935	1.000000	False
5		comoving_full_minus_odd	-0.023136	-0.003139	0.039091	0.810945	False

	model	representation	feature_space	mean_roc_auc	null_empirical_p
0	logistic	raw	full_4D	0.621956	0.004975
1	logistic	comoving	full_4D	0.504151	0.512438
2	rbf_svm	raw	full_4D	0.557226	0.074627
3	rbf_svm	raw	odd_net_displacement	0.565286	0.054726
4	rbf_svm	comoving	full_4D	0.348291	1.000000
5	rbf_svm	comoving	odd_net_displacement	0.371427	0.810945

Figure 3. Reversal decomposition, nonlinear RBF sensitivity, fold diagnostics, RBF paired-null summary, and final Phase 7 numerical ledger.

[↑ Back to Contents](#)

8. Cells 35–40: nonlinear RBF sensitivity and fold diagnostics

After the linear signal was explained by net displacement, one nonlinear model was allowed as a pre-specified sensitivity test. The question was whether nonlinear interactions in the full 4D sequence add information beyond the odd coordinate alone.

Cell 35 – Nonlinear sensitivity analysis

Type / role	Markdown / pre-specification
Code or content summary	Explained why an RBF classifier can detect nonlinear interactions such as odd×even terms and explicitly prohibited broad model searching or aggressive tuning.
Stored output	No executable output.
Interpretation / decision	Defined a single bounded nonlinear follow-up before seeing its null result.
Concepts taught / reinforced	Nonlinear decision boundaries; interaction structure; anti-cherry-picking design.

Cell 36 – Construct the RBF-SVM Pipeline

Type / role	Code / model definition
Code or content summary	Imported SVC and created StandardScaler → SVC(kernel="rbf", C=1.0, gamma="scale").
Stored output	VS Code rendered the Pipeline diagram.
Interpretation / decision	Used standard defaults rather than tuning to the observed dataset.
Concepts taught / reinforced	SVM; RBF kernel; regularization C; gamma; pipelines.

Cell 37 – Generic grouped score evaluator

Type / role	Code / reusable evaluation
Code or content summary	Extended the evaluator to use decision_function when available, otherwise predict_proba, while preserving GroupKFold and balanced accuracy.
Stored output	Function definition only.
Interpretation / decision	Allowed ROC AUC evaluation of SVC without probability calibration.
Concepts taught / reinforced	hasattr; decision_function; generic estimator interfaces.

Cell 38 – Observed nonlinear performance

Type / role	Code / RBF results
Code or content summary	Evaluated raw full, raw odd-only, co-moving full, and co-moving odd-only under identical grouped folds.

Stored output	Raw full AUC 0.557226, raw odd 0.565286; co-moving full 0.348291, co-moving odd 0.371427. Full did not outperform odd in either representation.
Interpretation / decision	No observed evidence that nonlinear full-sequence information beats net displacement.
Concepts taught / reinforced	Nonlinear sensitivity; controlled ablation comparison.

Cell 39 — Inspect nonlinear fold behavior

Type / role	Code / diagnostic
Code or content summary	Concatenated all RBF fold results into one table to determine whether low co-moving AUC came from one pathological fold.
Stored output	Co-moving full AUCs: 0.392085, 0.200288, 0.513314, 0.331690, 0.304076; four of five folds below 0.5. Odd-only showed a similar pattern.
Interpretation / decision	Below-chance co-moving behavior is distributed across folds, not caused by a single outlier fold.
Concepts taught / reinforced	pd.concat; assign; fold-level diagnostics.

Cell 40 — Quantify deviation from chance

Type / role	Code / descriptive diagnostic
Code or content summary	Added signed AUC-minus-chance and absolute deviation columns without changing the predefined success criterion.
Stored output	Raw full +0.057226; raw odd +0.065286; co-moving full -0.151709; co-moving odd -0.128573.
Interpretation / decision	Kept sub-0.5 values signed and explicitly rejected post-hoc inversion to 1-AUC.
Concepts taught / reinforced	Signed vs absolute deviations; post-hoc interpretation control.

NO POST-HOC AUC FLIPPING

An observed AUC of 0.348 was not reinterpreted as 0.652 by reversing class meaning after seeing the result. The predefined scientific question concerns correct-direction forward/reverse generalization, so below-chance values remain below chance.

[↑ Back to Contents](#)

9. Cells 41–44: RBF paired-null validation

The nonlinear model is judged with the same pair-preserving time-reversal null. Full-minus-odd AUC is also simulated directly so that the question of added full-sequence information has its own null distribution.

Cell 41 — Null validation of the nonlinear classifier

Type / role	Markdown / null specification
Code or content summary	Pre-specified four RBF analyses and the full-minus-odd contrast; stated that below-chance AUC would not be inverted.
Stored output	No executable output.
Interpretation / decision	Locked the nonlinear interpretation rules before running the null.
Concepts taught / reinforced	Pre-registration mindset; contrast statistics; null calibration.

Cell 42 — Run 200 RBF null replicates

Type / role	Code / simulation
Code or content summary	Repeated pair-orientation randomization with the same swaps for raw/comoving; built full and odd-only datasets; ran grouped RBF evaluation for all four analyses.
Stored output	RBF null replicates: 200; approximately 4000 model fits across 4 analyses × 5 folds × 200 replicates.
Interpretation / decision	Generated the appropriate finite-sample background for nonlinear performance and full-minus-odd contrasts.
Concepts taught / reinforced	Monte Carlo workload; nested validation; paired randomization.

Cell 43 — Compare observed RBF results with null

Type / role	Code / final nonlinear calibration
Code or content summary	Computed observed, null mean, q95, empirical upper-tail p, and above-q95 for six metrics.
Stored output	Raw full: 0.557226, p=0.074627. Raw odd: 0.565286, p=0.054726 and only ~0.000008 above estimated q95. Raw full-minus-odd: -0.008060, p=0.601990. Co-moving full: 0.348291, p=1.0. Co-moving odd: 0.371427, p=1.0. Co-moving full-minus-odd: -0.023136, p=0.810945.
Interpretation / decision	No convincing nonlinear evidence that full 4D carries extra time-direction information. The raw odd result is borderline only with respect to q95 and is not treated as robust.
Concepts taught / reinforced	Empirical tails; quantile instability at finite Monte Carlo size; conservative interpretation.

Cell 44 — Validate RBF null analysis

Type / role	Code / assertions
-------------	-------------------

Code or content summary	Checked 200 replicates, finite values, valid AUC ranges, and valid empirical p-values.
Stored output	RBF time-reversal null validation: PASSED.
Interpretation / decision	Closed the nonlinear sensitivity analysis without adding more models.
Concepts taught / reinforced	Simulation validation; stopping rule.

BORDERLINE MEANS BORDERLINE

The raw odd-only RBF observed AUC exceeded the estimated q95 by only about 8×10^{-6} , while the empirical upper-tail probability was ~ 0.055 . With only 200 null replicates, the conservative interpretation is “borderline, not robust,” not “significant.”

[↑ Back to Contents](#)

10. Cells 45–46: final numerical and scientific conclusion

Cell 45 – Final Phase 7 numerical summary

Type / role	Code / compact ledger
Code or content summary	Collected the main logistic and RBF AUC values and their empirical null probabilities into a single table.
Stored output	Logistic raw 0.621956, $p=0.004975$; logistic co-moving 0.504151, $p=0.512438$; RBF raw full 0.557226, $p=0.074627$; RBF raw odd 0.565286, $p=0.054726$; RBF co-moving full 0.348291, $p=1.0$; RBF co-moving odd 0.371427, $p=1.0$. Phase 7 computational analysis: PASSED.
Interpretation / decision	Produced the authoritative compact result set for README and handbook handoff.
Concepts taught / reinforced	Result ledgers; reproducible reporting; validation endpoint.

Cell 46 – Final Phase 7 conclusion

Type / role	Markdown / scientific conclusion
Code or content summary	Summarized the complete sequence construction, grouped validation, raw linear signal, odd/even decomposition, nonlinear sensitivity, null results, and strict interpretation boundary.
Stored output	No executable output.
Interpretation / decision	Concluded that Phase 7 did not provide convincing evidence for intrinsic two-step sequence-order irreversibility beyond common directional motion.
Concepts taught / reinforced	Scientific synthesis; negative-result interpretation; limitation statements.

FINAL PHASE 7 STATEMENT

Retaining the complete four-dimensional two-step sequence did not reveal convincing additional time-direction information beyond reversal-odd net displacement. The strong raw linear signal is predominantly a common directional-motion signal and largely disappears in the co-moving representation. This does not prove reversibility and does not justify entropy-production claims.

[↑ Back to Contents](#)

11. Integrated Phase 7 results and interpretation

11.1 Baseline linear classification

Representation	Mean ROC AUC	SD AUC	Mean balanced accuracy	Empirical null p
Raw	0.621956	0.084941	0.614395	0.004975
Co-moving	0.504151	0.080743	0.519994	0.512438

The raw linear classifier is genuinely above the paired null, whereas co-moving is indistinguishable from chance. This establishes detectable directional information in raw sequences under the linear model, but not its mechanism.

11.2 Reversal-parity decomposition

Representation	Feature space	Mean ROC AUC	Mean balanced accuracy
Raw	Full 4D	0.621956	0.614395
Raw	Odd net displacement	0.621925	0.614395
Raw	Even step difference	0.500000	0.500000
Co-moving	Full 4D	0.504151	0.519994
Co-moving	Odd net displacement	0.504087	0.519994
Co-moving	Even step difference	0.500000	0.500000

The near identity of full and odd-only performance means the linear model is essentially reading net two-frame displacement. The even coordinate is identical for forward and reversed examples and therefore sits exactly at chance, as required by symmetry.

11.3 Nonlinear sensitivity

Representation	Feature space	Observed AUC	Null q95	Empirical upper p	Full - odd?
Raw	Full 4D	0.557226	0.566600	0.074627	-0.008060
Raw	Odd net displacement	0.565286	0.565278	0.054726	—
Co-moving	Full 4D	0.348291	0.557063	1.000000	-0.023136
Co-moving	Odd net displacement	0.371427	0.565935	1.000000	—

The RBF model did not produce convincing forward-direction discrimination beyond its paired null, and full 4D did not outperform odd-only in either representation. The raw odd-only result sits on a Monte-Carlo boundary and is interpreted as borderline rather than robust.

11.4 Relation to Phase 6

Phase 6 found no reduced-feature KDE asymmetry beyond its finite-sample null. Phase 7 asked whether a richer representation could rescue a sequence-order signal. The answer is still negative with respect to intrinsic two-step ordering after common-motion correction: the only clear positive result is a raw linear directional signal that is mathematically explained by net displacement.

11.5 What the project can and cannot claim after Phase 7

WHAT PHASE 7 CAN SAY

Raw two-step data contain reproducible linear forward/reverse information relative to the paired null. That information is almost entirely captured by two-frame net displacement and is strongly reduced in co-moving coordinates. The tested full-

sequence nonlinear model does not add convincing information beyond net displacement.

WHAT PHASE 7 CANNOT SAY

It cannot identify common motion uniquely as microscope drift; it cannot prove the cells are time-reversible; it cannot exclude irreversibility at longer memory scales or in other observables; and it cannot support entropy-production or thermodynamic-arrow estimates from the present evidence.

[↑ Back to Contents](#)

12. Decision logic and project mindset

Decision point	Decision taken	Why
Phase 6 reduced representation was null-like	Test the complete 4D sequence rather than declare the system reversible.	A reduced feature map can discard information.
4D KDE would be data-hungry	Use classification instead of direct 4D density estimation.	Avoid curse-of-dimensionality problems with only 392 sequences.
Repeated sequences come from the same cells	Group cross-validation by cell_id.	Prevent biological leakage and memorization.
Raw logistic AUC was positive	Calibrate with a paired orientation null before interpreting.	Positive observed AUC alone is not evidence.
Raw linear null was strongly positive	Decompose by reversal parity before claiming sequence-order learning.	A linear classifier can exploit simple net direction.
Full~odd and even=chance	Interpret the raw signal as net directional displacement.	This is the mathematically supported mechanism.
Could nonlinear interactions remain?	Run one pre-specified RBF-SVM sensitivity model.	Test a plausible alternative without model shopping.
Co-moving RBF AUC <0.5	Keep the signed result; do not flip to 1-AUC post hoc.	Avoid changing the success criterion after seeing the data.
RBF full did not beat odd	Stop the ML search.	More models would primarily increase result-chasing risk.
Thermodynamic link remains tempting	Defer entropy-production/TUR claims.	Current evidence does not isolate intrinsic time irreversibility.

PROJECT MINDSET

A good exploratory analysis is allowed to end with “the richer method still did not detect the hypothesized sequence-order signal.” The scientific value lies in identifying what the positive raw classifier actually measured and in refusing to overinterpret it.

[↑ Back to Contents](#)

13. Python, statistics, and machine-learning concepts learned

Concept	Phase 7 example	General lesson
Feature matrix X	784 rows × 4 sequence components	Rows are examples; columns are predictive features.
Target vector y	1=forward, 0=reversed	Supervised learning needs explicit labels.
Groups	cell_id duplicated for forward/reverse pairs	Grouping metadata can control validation without becoming a feature.
np.vstack	Stack forward and reversed examples	Vertical stacking grows the sample axis.
np.concatenate	Join labels and groups	One-dimensional arrays must align with X rows.
Array slicing	[:,0:2] and[:,2:4]	Slices select contiguous feature blocks efficiently.
Boolean mask	swap_mask	Masks support vectorized conditional replacement.
np.random.default_rng	Seeded paired nulls	Explicit RNG objects improve reproducibility and avoid hidden global state.
Pipeline	StandardScaler → estimator	Preprocessing and model fitting should live inside each CV fold.
StandardScaler	Fit only on training cells	Prevent preprocessing leakage.
LogisticRegression	Interpretable linear baseline	Start simple before adding nonlinearity.
GroupKFold	Five cell-level folds	Validation structure must match biological dependence.
ROC AUC	Ranking forward above reversed	Threshold-free discrimination metric.
Balanced accuracy	Average class-wise recall	Useful when class balance might change in extensions.
predict_proba	Forward-class probability for logistic model	Class 1 is selected by column[:,1].
decision_function	Continuous SVC score	AUC does not require calibrated probabilities.
hasattr	Choose score interface generically	Reusable code can adapt to estimator capabilities.
pd.concat + assign	Combine fold tables with metadata	Long-format summaries simplify diagnostics.
quantile(0.95)	Null 95th percentile	Monte Carlo thresholds are estimated, not exact constants.
Empirical p-value	(1 + exceedances)/(N+1)	Simulation tail probabilities should include finite-sample correction.
Ablation / decomposition	Full vs odd vs even	Mechanistic interpretation often requires controlled feature-space comparisons.

[↑ Back to Contents](#)

14. Reproducibility, testing, and Git workflow

Reproducibility element	Phase 7 implementation
Notebook	notebooks/ 07_phase7_full_sequence_classification_MSC01.ipynb
Environment	Conda environment `cell-irreversibility`; Python 3.11 series
Random seeds	Base RANDOM_SEED=2026; null RNGs derived deterministically as +1 and +2
Cross-validation	GroupKFold with five folds and cell_id grouping
Preprocessing	StandardScaler inside Pipeline; never fitted globally before CV
Linear null	200 paired orientation-randomization replicates
RBF null	200 paired orientation-randomization replicates
Tests	`python -m pytest -q` → 9 passed in 0.56 s before repository handoff
Branch workflow	Phase 7 developed on `phase-7-full-sequence-classification`, then prepared for PR/merge into main
README	Updated through Phase 7 with classification, null, decomposition, and RBF conclusions

14.1 Why no new unit tests were added during Phase 7

The existing automated test suite protects authoritative reusable trajectory-step logic. Most Phase 7 functions currently live in the notebook and were validated with explicit assertions and deterministic outputs. If sequence construction and reversal/classification helpers are promoted into `src/`, that refactor should be accompanied by dedicated unit tests rather than leaving notebook-only assertions as the long-term protection layer.

14.2 Recommended future refactor

Candidate helper	Potential module	Suggested test
build_two_step_sequences	src/steps.py or src/sequences.py	Missing-middle-frame and duplicate-frame cases; exact $t \rightarrow t+1 \rightarrow t+2$ continuity.
reverse_two_step_sequences	src/sequences.py	Shape validation and involution $R(R(S))=S$.
decompose_reversal_coordinates	src/sequences.py	Odd sign reversal and even invariance.
randomize_pair_orientation	src/metrics.py or src/nulls.py	Pair preservation under keep/swap masks.

[↑ Back to Contents](#)

15. Phase 8 handoff and next decision space

Phase 7 has answered the immediate “did Phase 6 throw away too much information?” question for two-step sequences. The full 4D representation did not reveal convincing extra order information beyond net displacement under the models and nulls tested. The next phase should therefore change temporal depth or scientific observable, not merely try more classifiers on the same representation.

Candidate Phase 8 direction	Scientific value	Main risk / requirement
Longer ordered sequences	Tests memory beyond two consecutive steps.	Sequence count falls rapidly; dimensionality grows.
Temporal-memory observables	Can target persistence, acceleration-like structure, or delayed correlations directly.	Must choose observables before seeing outcomes and derive their reversal parity.
Direction-invariant / common-motion-robust features	Separates internal temporal structure from net translation more explicitly.	Must avoid accidentally removing biologically meaningful collective motion.
Sequence models with stronger regularization	Could capture longer-history nonlinear structure without direct density estimation.	Dataset may be too small; grouped validation and nulls remain mandatory.
Additional independent dataset or replicate sequence	Tests whether findings generalize beyond MSC01.	Requires compatible tracking and calibration.

DO NOT JUMP TO EPR/TUR YET

Entropy-production and thermodynamic uncertainty relation analyses remain deferred. A defensible thermodynamic interpretation requires clearer evidence of intrinsic time-asymmetric dynamics than the current two-step MSC01 results provide.

15.1 Fresh-chat starting state

A fresh chat should begin by reading the Phase 0–4 Master Handbook plus the standalone Phase 5, Phase 6, and this Phase 7 handbook. It should treat the README as the concise public project summary and the notebooks as the executable source of truth. Before any new analysis, verify ``git status``, activate the ``cell-irreversibility`` environment, and rerun ``python -m pytest -q``.

[↑ Back to Contents](#)

Appendix A. Key Phase 7 code listings

A1. Exact two-step time reversal

```
def reverse_two_step_sequences(sequence_array: np.ndarray) -> np.ndarray:
    sequence_array = np.asarray(sequence_array, dtype=float)
    if sequence_array.ndim != 2 or sequence_array.shape[1] != 4:
        raise ValueError("Expected shape (n, 4).")
    return np.column_stack([
        -sequence_array[:, 2],
        -sequence_array[:, 3],
        -sequence_array[:, 0],
        -sequence_array[:, 1],
    ])

```

A2. Balanced forward/reversed dataset construction

```
def build_classification_dataset(forward_array, reverse_array, cell_ids):
    if forward_array.shape != reverse_array.shape:
        raise ValueError("Forward and reverse arrays must have identical shapes.")
    if len(cell_ids) != len(forward_array):
        raise ValueError("cell_ids length does not match the number of sequences.")
    X = np.vstack([forward_array, reverse_array])
    y = np.concatenate([
        np.ones(len(forward_array), dtype=int),
        np.zeros(len(reverse_array), dtype=int),
    ])
    groups = np.concatenate([cell_ids, cell_ids])
    return X, y, groups

```

A3. Linear baseline pipeline

```
baseline_model = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=2000)),
])

```

A4. Pair-preserving null transformation

```
def randomize_pair_orientation(forward_array, reverse_array, swap_mask):
    pseudo_forward = forward_array.copy()
    pseudo_reverse = reverse_array.copy()
    pseudo_forward[swap_mask] = reverse_array[swap_mask]
    pseudo_reverse[swap_mask] = forward_array[swap_mask]
    return pseudo_forward, pseudo_reverse

```

A5. Empirical upper-tail probability

```
def empirical_upper_pvalue(observed, null_values):
    null_values = np.asarray(null_values, dtype=float)
    return (1 + np.sum(null_values >= observed)) / (len(null_values) + 1)

```

A6. Reversal-odd / reversal-even decomposition

```
def decompose_reversal_coordinates(sequence_array):
    sequence_array = np.asarray(sequence_array, dtype=float)
    first_step = sequence_array[:, 0:2]
    second_step = sequence_array[:, 2:4]
    odd = first_step + second_step
    even = first_step - second_step
    return odd, even

```

A7. Pre-specified nonlinear sensitivity model

```
rbf_model = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("classifier", SVC(kernel="rbf", C=1.0, gamma="scale")),
])

```

[↑ Back to Contents](#)

Appendix B. Exact numerical result ledger

Analysis	Representation / feature	Observed / mean	Null mean	Null q95	Empirical upper p
Logistic	Raw full 4D	0.621956	0.500347	0.555552	0.004975
Logistic	Co-moving full 4D	0.504151	0.502024	0.559142	0.512438
Logistic contrast	Raw - co-moving	0.117805	0.001677	0.033821	0.004975
Logistic decomposition	Raw odd	0.621925	—	—	—
Logistic decomposition	Raw even	0.500000	—	—	—
Logistic decomposition	Co-moving odd	0.504087	—	—	—
Logistic decomposition	Co-moving even	0.500000	—	—	—
RBF-SVM	Raw full 4D	0.557226	0.497950	0.566600	0.074627
RBF-SVM	Raw odd	0.565286	0.499204	0.565278	0.054726
RBF contrast	Raw full - odd	-0.008060	-0.001254	0.046895	0.601990
RBF-SVM	Co-moving full 4D	0.348291	0.498815	0.557063	1.000000
RBF-SVM	Co-moving odd	0.371427	0.501954	0.565935	1.000000
RBF contrast	Co-moving full - odd	-0.023136	-0.003139	0.039091	0.810945

B.1 Fold-level logistic results

Fold	Raw AUC	Raw BA	Co-moving AUC	Co-moving BA
1	0.637478	0.613924	0.520429	0.556962
2	0.662554	0.772152	0.611601	0.645570
3	0.615220	0.576923	0.497041	0.461538
4	0.483892	0.500000	0.389944	0.410256
5	0.710634	0.608974	0.506739	0.525641

B.2 Fold-level RBF results

Representation	Feature	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Raw	Full	0.701410	0.414357	0.599934	0.487179	0.583251
Raw	Odd	0.686188	0.402980	0.631986	0.504602	0.600674
Co-moving	Full	0.392085	0.200288	0.513314	0.331690	0.304076
Co-moving	Odd	0.435187	0.209422	0.564267	0.336785	0.311473

[↑ Back to Contents](#)

Appendix C. Glossary and notation

Term / symbol	Meaning in Phase 7
S	Observed ordered two-step sequence [dx1, dy1, dx2, dy2].
R(S)	Correct time reverse [-dx2, -dy2, -dx1, -dy1].
X	Classifier feature matrix.
y	Binary labels: forward=1, reversed=0.
groups	cell_id array used only for GroupKFold splitting.
GroupKFold	Cross-validation that keeps all examples from the same cell in one fold.
Leakage	Information from the test set entering model fitting or preprocessing.
StandardScaler	Transformer using training-set mean and standard deviation for feature scaling.
Logistic regression	Linear probabilistic classifier used as the interpretable baseline.
ROC AUC	Threshold-free ranking metric; 0.5 is chance in the predefined direction.
Balanced accuracy	Average recall across the two classes.
Paired null	Randomly swaps temporal orientation within each exact S/R(S) pair.
$o = d1 + d2$	Reversal-odd coordinate; equals net two-frame displacement.
$e = d1 - d2$	Reversal-even coordinate; unchanged under time reversal.
RBF-SVM	Support-vector classifier with radial-basis-function kernel.
decision_function	Continuous SVC score used to calculate ROC AUC.
q95	Estimated 95th percentile of the Monte Carlo null distribution.
empirical upper p	One-sided finite-simulation tail probability with +1 correction.
Common motion	Population-shared translation removed in the co-moving representation; not uniquely identifiable as technical drift.
Intrinsic sequence-order signal	Time-direction information beyond simple common directional displacement in the tested two-step sequence.

[↑ Back to Contents](#)

Appendix D. New-chat handoff prompt

COPY-PASTE HANDOFF PROMPT

Use the following prompt when continuing in a fresh chat. Attach the Phase 0–4 Master Handbook plus the Phase 5, Phase 6, and Phase 7 standalone handbooks if possible.

You are continuing a reproducible educational computational-biophysics project called `cell-irreversibility` (GitHub: <https://github.com/AbolDpirate/cell-irreversibility>). Read the attached handbooks as the project source of truth.

Current validated state through Phase 7:

- Dataset: Cell Tracking Challenge Fluo-C2DL-MSD, sequence 01; calibration 0.3 $\mu\text{m}/\text{pixel}$ and 20 min/frame.
- Primary clean data: 35 cells.
- Exact-lag displacement counts: $\tau_1=433$, $\tau_2=398$, $\tau_4=343$.
- Raw and co-moving representations are retained in parallel; common motion cannot be uniquely called microscope drift.
- Phase 6 constructed 392 exact two-step sequences from 32 cells and found no reduced-feature sequence asymmetry above its time-reversal-symmetrized null.
- Phase 7 retained the full 4D sequence $S=(dx_1, dy_1, dx_2, dy_2)$, with $R(S)=(-dx_2, -dy_2, -dx_1, -dy_1)$.
- Classifier dataset: 392 forward + 392 reversed = 784 examples, grouped by 32 biological cells with 5-fold GroupKFold.
- Logistic regression: raw mean AUC 0.621956 with paired-null $p=0.004975$; co-moving mean AUC 0.504151 with $p=0.512438$.
- Reversal decomposition: $\text{odd}=d_1+d_2$ (net two-frame displacement) and $\text{even}=d_1-d_2$. Raw full AUC 0.621956 $\approx$ raw odd AUC 0.621925; even-only AUC exactly 0.5. Co-moving full 0.504151 $\approx$ odd 0.504087.
- Therefore the significant raw linear signal is essentially net directional displacement, not richer two-step ordering.
- One pre-specified nonlinear sensitivity model was used: RBF-SVM. Raw full AUC 0.557226 ($p=0.074627$), raw odd 0.565286 ($p=0.054726$), full-minus-odd $p=0.601990$. Co-moving full 0.348291 and odd 0.371427; both empirical upper-tail $p=1.0$; do not invert below-chance AUC post hoc.
- Phase 7 conclusion: no convincing evidence that full two-step sequence ordering carries time-direction information beyond reversal-odd net displacement/common directional motion. This does not prove reversibility and does not justify entropy-production or TUR claims.
- Automated tests before handoff: 9 passed.

Teaching style: explain Python, statistics/ML, mathematics, and biophysics together; do not give unexplained copy-paste code. Keep scientific interpretation conservative and explicitly distinguish directional asymmetry from thermodynamic irreversibility. Prefer one notebook per phase, compact architecture, and no result chasing.

Before Phase 8: verify the repository is on clean `main`, activate `cell-irreversibility`, run `python -m pytest -q`, then design Phase 8 before coding. Phase 8 should change temporal depth or observable (e.g., longer sequences or pre-specified temporal-memory features), not simply try more classifiers on the same two-step representation.

[↑ Back to Contents](#)